

Wydział Informatyki Politechniki Białostockiej Przedmiot: Systemy operacyjne	Data: 18.03.2026r.
Temat: Projekt 1 – interfejs wywołań systemowych Linuksa. Grupa: PS8 Imię i nazwisko: Jakub Matusiewicz, Jakub Borkowski	Prowadzący: mgr inż. Tomasz Kuczyński

1 Cel projektu

Celem projektu było stworzenie demona poszukującego plików, który w określonych odstępach czasu rekurencyjnie skanuje system plików w poszukiwaniu ścieżek dopasowanych do zadanych wzorców. Oprócz podstawowej funkcjonalności przeszukiwania i omijania plików bez uprawnień, celem było zaimplementowanie:

- **Współbieżności:** wykorzystania procesów potomnych (jeden proces roboczy na każdy podany argument) oraz procesu nadzorczego do zarządzania skanowaniem.
- **Komunikacji asynchronicznej:** obsługi sygnałów systemowych POSIX (**SIGUSR1**, **SIGUSR2**) do natychmiastowego wybudzania demona, restartowania lub przerywania trwającego przeszukiwania.
- **Szczegółowego raportowania:** zapisywania wyników wyszukiwania oraz opcjonalnych, wyczerpujących informacji o stanie procesu (tryb *verbose*) do logu systemowego (**syslog**).

2 Treść zadania

Temat 2 - Demon(y) poszukujący plików

[20p] Demon(y) poszukujący plików. Program otrzymuje listę argumentów, z których każde jest fragmentem nazwy pliku. Program staje się demonem. Co kilkadziesiąt sekund (domyślny czas można zmienić za pomocą opcjonalnego parametru wiersza poleceń), rekurencyjnie skanuje system plików poszukując plików lub katalogów w których występuje zadany fragment nazwy. Pliki, do których demon nie ma praw dostępu są pomijane. Również takie katalogi są pomijane, dodatkowo katalog bez prawa odczytu i wykonania nie jest brany pod uwagę na etapie przeszukania rekurencyjnego. Nazwy odnalezionych plików lub katalogów są przekazywane zapisywane do logu systemowego (**syslog**). Umieszczone są w nim następujące informacje: pełna data, pełna ścieżka pliku, poszukiwany wzorzec. Odebranie sygnału **SIGUSR1** powoduje natychmiastowe

rozpoczęcie skanowania, gdy demon śpi. Dodatkowa opcja **-v** powoduje przesyłanie do logu wyczerpującej informacji o każdej z następujących czynności demona: a) uśpienie, b) obudzenie się, c) odbiór sygnału, d) porównanie nazwy pliku ze ścieżką.

Dodatkowo:

- a) [6p] Odebranie **SIGUSR1**, w chwili gdy trwa przeszukiwanie powoduje natychmiastowy restart przeszukiwania. Odebranie **SIGUSR2**, w chwili gdy trwa przeszukiwanie powoduje natychmiastowe zakończenie przeszukiwania i ponowne uśpienie demona.
- b) [8p] Przeszukiwanie jest prowadzone współbieżnie przez liczbę procesów równą liczbie argumentów. Dodatkowo występuje proces nadzorczy (śpi w oczekiwaniu na sygnały lub na zakończenie któregoś z procesów przeszukujących). Wysłanie procesowi nadzorczemu sygnału **SIGUSR1** albo **SIGUSR2** powoduje jego przekazania wszystkim procesom potomnym.

3 Opis poszczególnych modułów programu

Architektura projektu została podzielona na cztery główne, logiczne moduły funkcjonalne. Taki podział zapewnia czytelność i pozwala na separację mechanizmów systemowych (takich jak obsługa sygnałów i procesów) od logiki biznesowej (skanowanie plików).

3.1 Moduł Inicjalizacji, Konfiguracji i Głównej Pętli (**main**, **args**, **daemonize**, **state**)

Moduł ten odpowiada za punkt wejścia do programu, wczytanie konfiguracji oraz transformację procesu w demona systemowego.

- **Opis działania:** Moduł parsuje argumenty z wiersza poleceń (**args.c**), w tym wzorce poszukiwań, opcjonalny czas uśpienia i flagę **-v** (verbose). Następnie wykonuje procedurę odpięcia programu od terminala (**daemonize.c**), powołując do życia demona. Proces demonizacji został dodatkowo zabezpieczony przed wielokrotnym niezamierzonym uruchomieniem za pomocą pliku PID z blokadą systemową (**fcntl** na **/tmp/demonsearch.pid**). W pliku **state.h** zdefiniowano współdzielone struktury konfiguracyjne i stałe czasowe (runtime), z których korzysta główna pętla w **main.c**.
- **Kluczowe zmienne globalne:** Struktura konfiguracyjna przechowująca listę argumentów (wzorców) do wyszukiwania, czas uśpienia oraz flagę trybu *verbose*.

3.2 Moduł Nadzorcy i Współbieżności (**supervisor**, **proc_table**, **worker**)

Moduł realizujący wymóg współbieżnego przeszukiwania na podstawie procesów potomnych oraz realizujący logikę procesu nadzorczego (**supervisor**).

- **Opis działania:** Plik **supervisor.c** implementuje orkiestrację procesu nadzorczego. Na podstawie ilości argumentów powoływane są procesy robocze (**workery**), realizowane w pliku **worker.c**. Każdy z nich odpowiada za jeden cykl skanowania dla przypisanego mu wzorca. Stan procesów (cykl życia, PID) jest rejestrowany i utrzymywany w strukturach modułu **proc_table.c**, co ułatwia zarządzanie i ewentualne zabijanie/restartowanie **workerów**.
- **Kluczowe funkcje:** Funkcje monitorujące procesy potomne (**wait/waitpid**) oraz rozsyłające z procesu nadzorczego do **workerów** otrzymane sygnały (**SIGUSR1/SIGUSR2**).

3.3 Moduł Skanowania Systemu Plików i Raportowania (**scanner**, **match**, **logger**)

Moduł realizujący główną logikę operacji na plikach (I/O) i generowania logów.

- **Opis działania:** W module **scanner.c** zaimplementowano rekurencyjny bezpieczny traversal po systemie plików z uwzględnieniem sprawdzania praw dostępu do katalogów i plików. **match.c** służy jako moduł pomocniczy weryfikujący dopasowanie fragmentu nazwy pliku do zadanych wzorców z użyciem np. **strstr**. Architektura skanera została odseparowana od logiki raportowania przy użyciu mechanizmu wskaźników na funkcje (callbacks). Dzięki temu kod przechodzący po grafie katalogów operuje niezależnie od tego, co proces roboczy ostatecznie robi ze znalezioną ścieżką. Wyniki tych poszukiwań są ostatecznie wysyłane do systemowego logu przez specjalnie przygotowane funkcje z **logger.c**, które filtrują logi na standardowe oraz te dla opcji wyczerpującej (**-v**).

3.4 Moduł Sygnałów i Usypiania (**signals**, **sleep_control**)

Moduł odpowiadający za komunikację asynchroniczną i sterowanie czasem wykonania programu za pomocą sygnałów POSIX.

- **Opis działania:** **signals.c** konfiguruje handlers dla sygnałów **SIGUSR1** (budzenie/restart skanowania) i **SIGUSR2** (przerwanie skanowania i uśpienie). Zmieniają one globalne flagi kontrolne. Z kolei **sleep_control.c** używa tych flag do zarządzania interwałami snu nadzorcy i jego przedwczesnym wybudzaniem, wykorzystując bezpieczne dla systemu przerwanie funkcji **clock_nanosleep()**.

- **Kluczowe zmienne globalne:** Atomowe flagi systemowe (`volatile sig_atomic_t`), na których opiera się asynchroniczne przerywanie mechanizmu spania (obsługa błędu `EINTR`), powiadamiające demona, czy ma natychmiast przestać szukać czy zrestartować skanowanie.

4 Zrzuty ekranu działającego programu (demonstracja)

W celu zaprezentowania poprawności działania programu, poniżej przedstawiono kilka kluczowych scenariuszy testowych, które zostały przeprowadzone na działającym demonie poszukującym plików. Każdy z nich ilustruje różne aspekty funkcjonalności, od standardowego działania po obsługę sygnałów i trybu verbose.

4.1 Walidacja argumentów i interfejs wiersza poleceń

Pierwszym etapem działania aplikacji jest parsowanie argumentów przekazanych przez użytkownika. Aby zapewnić stabilność demona i zapobiec nieprzewidzianemu zachowaniu systemu na skutek nieprawidłowych danych wejściowych, zaimplementowano rygorystyczny mechanizm weryfikacji.

Na Rysunku 1 zaprezentowano reakcję programu na niepoprawne wywołania. Aplikacja skutecznie wykrywa brak wymaganych wzorców wyszukiwania, brak wartości dla opcji wymagających parametru (np. czas interwału dla flagi `-i`), a także próby wielokrotnego zdefiniowania tej samej ścieżki (podwójna flaga `-d`). W każdym z tych przypadków program bezpiecznie przerywa działanie, wyświetlając komunikat błędu oraz wskazówkę dotyczącą użycia flagi pomocy (`-h`), która prezentuje pełną instrukcję obsługi.

```

$ demonsearch
Error: at least one search pattern must be provided.
Try 'demonsearch --help' for more information.
$ demonsearch -v
Error: at least one search pattern must be provided.
Try 'demonsearch --help' for more information.
$ demonsearch -i
Error: Option '-i' requires an argument.
Try 'demonsearch --help' for more information.
$ demonsearch -d /tmp/ -d ~/test/
Error: --dir option specified multiple times.
Try 'demonsearch --help' for more information.
$ demonsearch -h
Usage: demonsearch [-v|--verbose] [-t|--time seconds] [-d|--dir path] [--help] <pattern1> [pattern2 ...]

Options:
  -t, --time, -i, --interval <sec>  Set sleep interval between scans (default 60)
  -d, --dir <path>                    Set starting directory (default: root '/')
  -v, --verbose                        Start in verbose mode (logs daemon events)
  -h, --help                          Show this help message and exit

```

Rysunek 1: Zrzut ekranu przedstawiający obsługę błędnych argumentów oraz pełny interfejs pomocy demona.

4.2 Standardowe uruchomienie z kilkoma wzorcami

Demon został wywołany z pięcioma poszukiwanymi wzorcami. Komenda **ps u -C demonsearch** (Rysunek 2) potwierdza proces demonizacji i utworzenie dokładnie pięciu podprocesów powiązanych z procesem nadzorczym. Dodatkowo ujęto scenariusz pokazujący wysłanie sygnału zamknięcia (**kill**) do samego nadzorcy (supervisor), co skutkowało całkowitym wyczyszczeniem środowiska (**ps u** wykazał puste zwrócone linie), co dowodzi o bezpiecznym sprzątnięciu i powiązaniu hierarchicznym procesów.

```
> demonsearch BysKfoNU8n eLd2rVMshz LSawV60AyU Q0irEFD4lJ W82dWffH14
> ps u -C demonsearch
USER          PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
knrt      950292   0.0  0.0   2720  1416 ?        S   02:40   0:00 demonsearch BysKfoNU8n eLd2rVMshz LSawV60AyU Q0irEFD4lJ W82dWffH14
knrt      950293  58.3  0.0   3168  1756 ?        R   02:40   0:03 demonsearch BysKfoNU8n eLd2rVMshz LSawV60AyU Q0irEFD4lJ W82dWffH14
knrt      950294  58.3  0.0   3104  1744 ?        R   02:40   0:03 demonsearch BysKfoNU8n eLd2rVMshz LSawV60AyU Q0irEFD4lJ W82dWffH14
knrt      950295  58.8  0.0   3104  1756 ?        R   02:40   0:03 demonsearch BysKfoNU8n eLd2rVMshz LSawV60AyU Q0irEFD4lJ W82dWffH14
knrt      950296  59.0  0.0   3072  1756 ?        R   02:40   0:03 demonsearch BysKfoNU8n eLd2rVMshz LSawV60AyU Q0irEFD4lJ W82dWffH14
knrt      950297  58.6  0.0   3072  1740 ?        R   02:40   0:03 demonsearch BysKfoNU8n eLd2rVMshz LSawV60AyU Q0irEFD4lJ W82dWffH14
> kill 950292
> ps u -C demonsearch
USER          PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
```

Rysunek 2: Lista uruchomionych procesów demona oraz weryfikacja ich kaskadowego zamknięcia po uśmierceniu nadzorcy.

Standardowe wyjście w systemowym dzienniku (Rysunek 3) potwierdza, że logowany jest czas, pełna ścieżka i powiązany z procesem na sztywno wzorec (bez użycia PID'ów w treści, co eliminuje nieczytelność po restartach):

```
> journalctl -t demonsearch -f --since "now"
Apr 30 02:40:15 knrt-laptop demonsearch[950293]: component=worker event=match timestamp=2026-04-30T02:40:15+0200 pattern="BysKfoNU8n" path="/home/knrt/test/BysKfoNU8n"
Apr 30 02:40:15 knrt-laptop demonsearch[950293]: component=worker event=match timestamp=2026-04-30T02:40:15+0200 pattern="LSawV60AyU" path="/home/knrt/test/LSawV60AyU"
Apr 30 02:40:15 knrt-laptop demonsearch[950294]: component=worker event=match timestamp=2026-04-30T02:40:15+0200 pattern="Q0irEFD4lJ" path="/home/knrt/test/Q0irEFD4lJ"
Apr 30 02:40:15 knrt-laptop demonsearch[950297]: component=worker event=match timestamp=2026-04-30T02:40:15+0200 pattern="W82dWffH14" path="/home/knrt/test/W82dWffH14"
Apr 30 02:40:15 knrt-laptop demonsearch[950294]: component=worker event=match timestamp=2026-04-30T02:40:15+0200 pattern="eLd2rVMshz" path="/home/knrt/test/eLd2rVMshz"
Apr 30 02:41:55 knrt-laptop demonsearch[950293]: component=worker event=match timestamp=2026-04-30T02:41:55+0200 pattern="BysKfoNU8n" path="/home/knrt/test/BysKfoNU8n"
Apr 30 02:41:55 knrt-laptop demonsearch[950297]: component=worker event=match timestamp=2026-04-30T02:41:55+0200 pattern="W82dWffH14" path="/home/knrt/test/W82dWffH14"
Apr 30 02:41:55 knrt-laptop demonsearch[950294]: component=worker event=match timestamp=2026-04-30T02:41:55+0200 pattern="eLd2rVMshz" path="/home/knrt/test/eLd2rVMshz"
Apr 30 02:41:55 knrt-laptop demonsearch[950295]: component=worker event=match timestamp=2026-04-30T02:41:55+0200 pattern="LSawV60AyU" path="/home/knrt/test/LSawV60AyU"
```

Rysunek 3: Zrzut wpisów z dziennika po dopasowaniu plików do wzorców.

4.3 Szczegółowe monitorowanie skanowania z flagą -v (verbose)

Uruchomienie programu z opcją **-v** wymusza logowanie wyczerpujących informacji o każdym kroku wykonywanym przez demona, w tym o procesie analizy plików. Zrzut ekranu (Rysunek 4) ilustruje przykładowe polecenie testowe uruchamiające skanowanie wybranych wzorców z włączonym trybem *verbose*.

```

> ls -la ~/test
total 20
drwxr-xr-x  3 knrt knrt  4096 Apr 25 22:01 .
drwx----- 122 knrt knrt 12288 Apr 30 02:48 ..
-rw-r--r--  1 knrt knrt    0 Apr 22 15:00 0ysKfoNU8n
-rw-r--r--  1 knrt knrt    0 Apr 22 15:00 ajcaMkgZrp
-rw-r--r--  1 knrt knrt    0 Apr 15 10:31 asdf
-rw-r--r--  1 knrt knrt    0 Apr 22 15:00 eLd2rVMshz
-rw-r--r--  1 knrt knrt    0 Apr 15 10:34 fdsa
-rw-r--r--  1 knrt knrt    0 Apr 22 15:00 KcNs03tMjk
-rw-r--r--  1 knrt knrt    0 Apr 22 15:00 LSawV60AyU
-rw-r--r--  1 knrt knrt    0 Apr 22 15:00 nASSLCdApL
-rw-r--r--  1 knrt knrt    0 Apr 22 15:00 pcxx0CI0jP
-rw-r--r--  1 knrt knrt    0 Apr 22 15:00 Q0irEFD4lJ
-rw-r--r--  1 knrt knrt    0 Apr 22 15:00 rQ66lnEsCu
drwxr-xr-x  2 knrt knrt  4096 Apr 25 22:01 test2
-rw-r--r--  1 knrt knrt    0 Apr 22 15:00 W82dWffHi4
-rw-r--r--  1 knrt knrt    0 Apr 25 21:50 'zly'$'\n''plik'$'\033''''wirus'.txt'
> mkdir ~/test/zablokowany_folder
> chmod 000 ~/test/zablokowany_folder
> touch ~/test/zablokowany_plik
> chmod 000 ~/test/zablokowany_plik
> demonsearch -v -i 120 -d ~/test/ LSawV60AyU Q0irEFD4lJ W82dWffHi4

```

Rysunek 4: Terminal pokazujący uruchomienie testowe programu demonsearch z włączoną flagą verbose.

Potwierdzeniem prawidłowego działania tego trybu są wpisy w dzienniku systemowym. Jak widać na Rysunku 5, tryb *verbose* generuje ogromną ilość logów dla zdarzenia skanowania. Widoczny jest moment startu procesu (event=*start*), rozpoczęcia przeszukiwania, a przede wszystkim logi typu *compare* informujące o ewaluacji każdej pojedynczej napotkanej ścieżki (z wynikiem *matched=true/false*). Dopiero po całkowitym przeanalizowaniu drzewa plików, worker loguje przejście w stan uśpienia (event=*state_change*, state=*sleeping*).

```

$ sudo -i -c demomsearch -f -s since "now"
Apr 30 02:51:30 knrt-Laptop demomsearch[970336]: component-supervisor event-start target_dir="/home/knrt/test" worker_count=3
Apr 30 02:51:30 knrt-Laptop demomsearch[970337]: component-worker event-start pattern="LSawV60AYu"
Apr 30 02:51:30 knrt-Laptop demomsearch[970337]: component-worker event-state_change state=scanning pattern="LSawV60AYu"
Apr 30 02:51:30 knrt-Laptop demomsearch[970337]: component-worker event-scan_start pattern="LSawV60AYu"
Apr 30 02:51:30 knrt-Laptop demomsearch[970336]: component-supervisor eventwait_loop_enter total_workers=3
Apr 30 02:51:30 knrt-Laptop demomsearch[970337]: component-worker event-copare path="/home/knrt/test/q64nE8Cu" pattern="LSawV60AYu" matched=false
Apr 30 02:51:30 knrt-Laptop demomsearch[970337]: component-worker event-copare path="/home/knrt/test/0yKfoNU8u" pattern="LSawV60AYu" matched=false
Apr 30 02:51:30 knrt-Laptop demomsearch[970337]: component-worker event-copare path="/home/knrt/test/zlyVnpl3kXid1wirus\".txt" pattern="LSawV60AYu" matched=false
Apr 30 02:51:30 knrt-Laptop demomsearch[970337]: component-worker event-copare path="/home/knrt/test/pcx0CIG9P" pattern="LSawV60AYu" matched=false
Apr 30 02:51:30 knrt-Laptop demomsearch[970337]: component-worker event-copare path="/home/knrt/test/el22rVmh2" pattern="LSawV60AYu" matched=false
Apr 30 02:51:30 knrt-Laptop demomsearch[970337]: component-worker event-copare path="/home/knrt/test/W82dWFFH14" pattern="LSawV60AYu" matched=false
Apr 30 02:51:30 knrt-Laptop demomsearch[970339]: component-worker event-start pattern="W82dWFFH14"
Apr 30 02:51:30 knrt-Laptop demomsearch[970337]: component-worker event-copare path="/home/knrt/test/LSawV60AYu" pattern="LSawV60AYu" matched=true
Apr 30 02:51:30 knrt-Laptop demomsearch[970338]: component-worker event-start pattern="Q0irEPD4LJ"
Apr 30 02:51:30 knrt-Laptop demomsearch[970339]: component-worker event-state_change state=scanning pattern="W82dWFFH14"
Apr 30 02:51:30 knrt-Laptop demomsearch[970339]: component-worker event-scan_start pattern="W82dWFFH14"
Apr 30 02:51:30 knrt-Laptop demomsearch[970337]: component-worker event-match timestamp=2026-04-30T02:51:30+0200 pattern="LSawV60AYu" path="/home/knrt/test/LSawV60AYu"
Apr 30 02:51:30 knrt-Laptop demomsearch[970337]: component-worker event-copare path="/home/knrt/test/Q0irEPD4LJ" pattern="LSawV60AYu" matched=false
Apr 30 02:51:30 knrt-Laptop demomsearch[970338]: component-worker event-state_change state=scanning pattern="Q0irEPD4LJ"
Apr 30 02:51:30 knrt-Laptop demomsearch[970339]: component-worker event-copare path="/home/knrt/test/test2" pattern="LSawV60AYu" matched=false
Apr 30 02:51:30 knrt-Laptop demomsearch[970338]: component-worker event-scan_start pattern="Q0irEPD4LJ"
Apr 30 02:51:30 knrt-Laptop demomsearch[970337]: component-worker event-copare path="/home/knrt/test/test2/zlyVnpl3kXid1wirus\".txt" pattern="LSawV60AYu" matched=false
Apr 30 02:51:30 knrt-Laptop demomsearch[970339]: component-worker event-copare path="/home/knrt/test/q64nE8Cu" pattern="W82dWFFH14" matched=false
Apr 30 02:51:30 knrt-Laptop demomsearch[970339]: component-worker event-copare path="/home/knrt/test/0yKfoNU8u" pattern="W82dWFFH14" matched=false
Apr 30 02:51:30 knrt-Laptop demomsearch[970337]: component-worker event-copare path="/home/knrt/test/Kc8B3tHj4" pattern="LSawV60AYu" matched=false
Apr 30 02:51:30 knrt-Laptop demomsearch[970339]: component-worker event-copare path="/home/knrt/test/zlyVnpl3kXid1wirus\".txt" pattern="W82dWFFH14" matched=false
Apr 30 02:51:30 knrt-Laptop demomsearch[970337]: component-worker event-copare path="/home/knrt/test/asset" pattern="LSawV60AYu" matched=false
Apr 30 02:51:30 knrt-Laptop demomsearch[970339]: component-worker event-copare path="/home/knrt/test/pcx0CIG9P" pattern="W82dWFFH14" matched=false
Apr 30 02:51:30 knrt-Laptop demomsearch[970337]: component-worker event-copare path="/home/knrt/test/fds4" pattern="LSawV60AYu" matched=false
Apr 30 02:51:30 knrt-Laptop demomsearch[970338]: component-worker event-copare path="/home/knrt/test/0yKfoNU8u" pattern="W82dWFFH14" matched=false
Apr 30 02:51:30 knrt-Laptop demomsearch[970337]: component-worker event-state_change state=sleeping pattern="LSawV60AYu"
Apr 30 02:51:30 knrt-Laptop demomsearch[970337]: component-worker event-sleep pattern="LSawV60AYu" interval_sec=120
Apr 30 02:51:30 knrt-Laptop demomsearch[970339]: component-worker event-copare path="/home/knrt/test/W82dWFFH14" pattern="W82dWFFH14" matched=true
Apr 30 02:51:30 knrt-Laptop demomsearch[970338]: component-worker event-match timestamp=2026-04-30T02:51:30+0200 pattern="W82dWFFH14" path="/home/knrt/test/W82dWFFH14"
Apr 30 02:51:30 knrt-Laptop demomsearch[970339]: component-worker event-copare path="/home/knrt/test/q64nE8Cu" pattern="W82dWFFH14" matched=false
Apr 30 02:51:30 knrt-Laptop demomsearch[970339]: component-worker event-copare path="/home/knrt/test/LSawV60AYu" pattern="W82dWFFH14" matched=false
Apr 30 02:51:30 knrt-Laptop demomsearch[970338]: component-worker event-copare path="/home/knrt/test/0yKfoNU8u" pattern="Q0irEPD4LJ" matched=false
Apr 30 02:51:30 knrt-Laptop demomsearch[970339]: component-worker event-copare path="/home/knrt/test/Q0irEPD4LJ" pattern="W82dWFFH14" matched=false
Apr 30 02:51:30 knrt-Laptop demomsearch[970339]: component-worker event-copare path="/home/knrt/test/test2" pattern="W82dWFFH14" matched=false
Apr 30 02:51:30 knrt-Laptop demomsearch[970338]: component-worker event-copare path="/home/knrt/test/zlyVnpl3kXid1wirus\".txt" pattern="Q0irEPD4LJ" matched=false
Apr 30 02:51:30 knrt-Laptop demomsearch[970338]: component-worker event-copare path="/home/knrt/test/pcx0CIG9P" pattern="Q0irEPD4LJ" matched=false
Apr 30 02:51:30 knrt-Laptop demomsearch[970339]: component-worker event-copare path="/home/knrt/test/test2/zlyVnpl3kXid1wirus\".txt" pattern="W82dWFFH14" matched=false
Apr 30 02:51:30 knrt-Laptop demomsearch[970338]: component-worker event-copare path="/home/knrt/test/el22rVmh2" pattern="Q0irEPD4LJ" matched=false

```

Rysunek 5: Wyjście z logu systemowego demonstujące wyczerpujące operacje workera (logi compare) wymuszone przez tryb verbose.

4.4 Wymuszone zatrzymanie pojedynczego procesu roboczego

Architektura demona zakłada niezależne i współbieżne działanie procesów potomnych. Aby zweryfikować poprawność zarządzania cyklem życia workerów oraz reakcję nadzorcy na niespodziewaną utratę procesu, przeprowadzono test polegający na ręcznym wysłaniu sygnału zakończenia do jednego z nich.

Na Rysunku 6 zaprezentowano stan procesów demona przed i po wykonaniu polecenia kill. Po wysłaniu domyślnego sygnału zamknięcia do wybranego workera (w tym przypadku PID 970338), znika on z listy aktywnych procesów, podczas gdy nadzorca i pozostałe procesy potomne nieprzerwanie kontynuują swoją pracę.

```

$ ps u -C demomsearch
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
knrt      970336  0.0  0.0   2720  1424 ?        S    02:51   0:00 demomsearch -v -i 120 -d /home/knrt/test/ LSawV60AYu Q0irEPD4LJ W82dW
knrt      970337  0.0  0.0   2720  1436 ?        S    02:51   0:00 demomsearch -v -i 120 -d /home/knrt/test/ LSawV60AYu Q0irEPD4LJ W82dW
knrt      970338  0.0  0.0   2720  1436 ?        S    02:51   0:00 demomsearch -v -i 120 -d /home/knrt/test/ LSawV60AYu Q0irEPD4LJ W82dW
knrt      970339  0.0  0.0   2720  1420 ?        S    02:51   0:00 demomsearch -v -i 120 -d /home/knrt/test/ LSawV60AYu Q0irEPD4LJ W82dW
$ kill 970338
$ ps u -C demomsearch
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
knrt      970336  0.0  0.0   2720  1424 ?        S    02:51   0:00 demomsearch -v -i 120 -d /home/knrt/test/ LSawV60AYu Q0irEPD4LJ W82dW
knrt      970337  0.0  0.0   2720  1436 ?        S    02:51   0:00 demomsearch -v -i 120 -d /home/knrt/test/ LSawV60AYu Q0irEPD4LJ W82dW
knrt      970339  0.0  0.0   2720  1420 ?        S    02:51   0:00 demomsearch -v -i 120 -d /home/knrt/test/ LSawV60AYu Q0irEPD4LJ W82dW

```

Rysunek 6: Zrzut terminala ukazujący listę procesów demona przed i po ręcznym wyłączeniu pojedynczego workera.

Faktyczną mechanikę tego zdarzenia można zaobserwować w logach aplika-

cji (Rysunek 7). Wybrany worker przechwytuje sygnał 15 (SIGTERM) i wykonuje procedurę bezpiecznego wyłączenia, zwalniając przydzieloną pamięć (`event=shutdown`). Jednocześnie proces główny (nadzorca) natychmiast dowiaduje się o śmierci swojego potomka za sprawą sygnału 17 (SIGCHLD) wysłanego przez jądro systemu. Nadzorca prawidłowo obsługuje to przerwanie — aktualizuje swoją wewnętrzną tablicę procesów (`proc_table`) i odnotowuje zaktualizowaną liczbę aktywnych workerów (`remaining_workers=2`).

```
Apr 30 02:53:42 knrt-laptop demonsearch[970338]: component=worker event=signal_received pattern="Q0irEFD4LJ" signal=15
Apr 30 02:53:42 knrt-laptop demonsearch[970338]: component=worker event=shutdown pattern="Q0irEFD4LJ"
Apr 30 02:53:42 knrt-laptop demonsearch[970336]: component=supervisor event=signal_received signal=17
Apr 30 02:53:42 knrt-laptop demonsearch[970336]: component=supervisor event=worker_exit remaining_workers=2
```

Rysunek 7: Zapis z dziennika systemowego rejestrujący wymuszone zamknięcie workera (sygnał 15) oraz asynchroniczną reakcję nadzorcy (sygnał 17).

4.5 Asynchroniczna zmiana stanów i przerywanie skanowania (w locie)

Kolejnym testem weryfikującym poprawność wielobieżnej architektury demona było sprawdzenie jego reakcji na sekwencję sprzecznych poleceń. Celem było upewnienie się, że procesy robocze nie ignorują sygnałów, gdy są zajęte operacjami wejścia/wyjścia (I/O). Na Rysunku 8 przedstawiono ciąg poleceń powłoki. Za pomocą narzędzia `pgrep` pobrano identyfikator nadzorcy, a następnie wysłano do niego w sekundowych odstępach sygnał uśpienia (12 – SIGUSR2), sygnał wybudzenia/skanowania (10 – SIGUSR1) oraz ponownie sygnał uśpienia (12). Test zakończono komendą `killall`.

```
> demonsearch -v 0ysKfoNU8n eLd2rVMshz
> PID=$(pgrep -o demonsearch); kill -12 $PID; sleep 1; kill -10 $PID; sleep 1; kill -12 $PID
> killall demonsearch
```

Rysunek 8: Zrzut terminala z poleceniami wysyłającymi ręczną sekwencję sygnałów sterujących.

Analiza wygenerowanego dziennika (Rysunek 9) udowadnia poprawne działanie systemu. Nadzorca prawidłowo przekazał sygnały do workerów (`event=signal_forward`). Kluczowym zachowaniem jest zarejestrowany wpis `event=scan_interrupted`. Potwierdza on, że po wysłaniu sygnału SIGUSR2 proces roboczy nie oczekiwał na całkowite zakończenie przeszukiwania dysku, lecz bezpiecznie je przerwał, przechodząc w stan uśpienia (`event=state_change, state=sleeping`). Następujący bezpośrednio po nim sygnał wybudzenia prawidłowo wywołał zdarzenie `wakeup` i wznowił procedurę, po czym proces reagował na kolejne przerwania bez zawieszania się.

```

Apr 30 17:46:49 knrt-laptop demonsearch[183552]: component=worker event=compare path="/sys/class/pci_bus/0000:c3" pattern="GysKfoNU8n" matched=false
Apr 30 17:46:49 knrt-laptop demonsearch[183553]: component=worker event=compare path="/sys/class/rfkill" pattern="eLd2rVMshz" matched=false
Apr 30 17:46:49 knrt-laptop demonsearch[183552]: component=worker event=compare path="/sys/class/pci_bus/0000:03" pattern="GysKfoNU8n" matched=false
Apr 30 17:46:49 knrt-laptop demonsearch[183552]: component=worker event=compare path="/sys/class/pci_bus/0000:01" pattern="GysKfoNU8n" matched=false
Apr 30 17:46:49 knrt-laptop demonsearch[183553]: component=worker event=compare path="/sys/class/rfkill/rfkill0" pattern="eLd2rVMshz" matched=false
Apr 30 17:46:53 knrt-laptop demonsearch[183551]: component=supervisor event=signal_received signal=12
Apr 30 17:46:53 knrt-laptop demonsearch[183551]: component=supervisor event=signal_forward signal=12 target=workers
Apr 30 17:46:53 knrt-laptop demonsearch[183553]: component=worker event=scan_interrupted pattern="eLd2rVMshz"
Apr 30 17:46:53 knrt-laptop demonsearch[183552]: component=worker event=signal_received pattern="GysKfoNU8n" signal=12
Apr 30 17:46:53 knrt-laptop demonsearch[183553]: component=worker event=signal_received pattern="eLd2rVMshz" signal=12
Apr 30 17:46:53 knrt-laptop demonsearch[183552]: component=worker event=state_change state=sleeping pattern="GysKfoNU8n"
Apr 30 17:46:53 knrt-laptop demonsearch[183553]: component=worker event=state_change state=sleeping pattern="eLd2rVMshz"
Apr 30 17:46:53 knrt-laptop demonsearch[183553]: component=worker event=sleep pattern="eLd2rVMshz" interval_sec=60
Apr 30 17:46:53 knrt-laptop demonsearch[183552]: component=worker event=sleep pattern="GysKfoNU8n" interval_sec=60
Apr 30 17:46:54 knrt-laptop demonsearch[183551]: component=supervisor event=signal_received signal=10
Apr 30 17:46:54 knrt-laptop demonsearch[183551]: component=supervisor event=signal_forward signal=10 target=workers
Apr 30 17:46:54 knrt-laptop demonsearch[183553]: component=worker event=wakeup pattern="eLd2rVMshz" reason=sigusr1
Apr 30 17:46:54 knrt-laptop demonsearch[183552]: component=worker event=wakeup pattern="GysKfoNU8n" reason=sigusr1
Apr 30 17:46:54 knrt-laptop demonsearch[183553]: component=worker event=signal_received pattern="eLd2rVMshz" signal=10
Apr 30 17:46:54 knrt-laptop demonsearch[183552]: component=worker event=signal_received pattern="GysKfoNU8n" signal=10
Apr 30 17:46:54 knrt-laptop demonsearch[183553]: component=worker event=state_change state=scanning pattern="eLd2rVMshz"
Apr 30 17:46:54 knrt-laptop demonsearch[183553]: component=worker event=state_change state=scanning pattern="GysKfoNU8n"
Apr 30 17:46:54 knrt-laptop demonsearch[183552]: component=worker event=scan_start pattern="eLd2rVMshz"
Apr 30 17:46:54 knrt-laptop demonsearch[183552]: component=worker event=scan_start pattern="GysKfoNU8n"
Apr 30 17:46:55 knrt-laptop demonsearch[183551]: component=supervisor event=signal_received signal=12
Apr 30 17:46:55 knrt-laptop demonsearch[183551]: component=supervisor event=signal_forward signal=12 target=workers
Apr 30 17:46:55 knrt-laptop demonsearch[183553]: component=worker event=scan_interrupted pattern="eLd2rVMshz"
Apr 30 17:46:55 knrt-laptop demonsearch[183553]: component=worker event=signal_received pattern="eLd2rVMshz" signal=12
Apr 30 17:46:55 knrt-laptop demonsearch[183553]: component=worker event=state_change state=sleeping pattern="eLd2rVMshz"
Apr 30 17:46:55 knrt-laptop demonsearch[183553]: component=worker event=sleep pattern="eLd2rVMshz" interval_sec=60
Apr 30 17:46:55 knrt-laptop demonsearch[183552]: component=worker event=scan_interrupted pattern="GysKfoNU8n"
Apr 30 17:46:55 knrt-laptop demonsearch[183552]: component=worker event=signal_received pattern="GysKfoNU8n" signal=12
Apr 30 17:46:55 knrt-laptop demonsearch[183552]: component=worker event=state_change state=sleeping pattern="GysKfoNU8n"
Apr 30 17:46:55 knrt-laptop demonsearch[183552]: component=worker event=sleep pattern="GysKfoNU8n" interval_sec=60
Apr 30 17:47:02 knrt-laptop demonsearch[183551]: component=supervisor event=signal_received signal=15
Apr 30 17:47:02 knrt-laptop demonsearch[183551]: component=supervisor event=shutdown_requested signal=15 action=terminate_workers
Apr 30 17:47:02 knrt-laptop demonsearch[183553]: component=worker event=wakeup pattern="eLd2rVMshz" reason=other
Apr 30 17:47:02 knrt-laptop demonsearch[183552]: component=worker event=wakeup pattern="GysKfoNU8n" reason=other
Apr 30 17:47:02 knrt-laptop demonsearch[183551]: component=supervisor event=cleanup_wait
Apr 30 17:47:02 knrt-laptop demonsearch[183553]: component=worker event=signal_received pattern="eLd2rVMshz" signal=15
Apr 30 17:47:02 knrt-laptop demonsearch[183552]: component=worker event=signal_received pattern="GysKfoNU8n" signal=15
Apr 30 17:47:02 knrt-laptop demonsearch[183553]: component=worker event=shutdown pattern="eLd2rVMshz"
Apr 30 17:47:02 knrt-laptop demonsearch[183552]: component=worker event=shutdown pattern="GysKfoNU8n"
Apr 30 17:47:02 knrt-laptop demonsearch[183551]: component=supervisor event=shutdown_complete

```

Rysunek 9: Zapis dziennika pokazujący asynchroniczne przerywanie trwającego skanowania oraz płynne zmiany stanów wywołane poleceniami użytkownika.

5 Podział pracy

- **Jakub Borkowski:** Implementacja odpięcia procesu (**daemonize**), parsowania argumentów (**args**), procedury bezpiecznego i rekurencyjnego skanowania drzewa katalogów (**scanner**, **match**) oraz logowania do systemowego dziennika (**logger**).
- **Jakub Matusiewicz:** Pisanie sprawozdania, opracowanie obsługi sygnałów **SIGUSR1/SIGUSR2** i flag współdzielonych (**signals**), implementacja procesu nadzorczego i tabeli procesów do zarządzania potomkami (**supervisor**, **proc_table**, **worker**) oraz kontrola wybudzania/usypiania głównej pętli (**sleep_control**).